



دانشگاه بوعلی سینا
دانشگاه بوعلی سینا
Bu-Ali Sina University

HoodQuest

پروژه پایانی درس ساختمان داده و الگوریتم ها

استاد درس : استاد جاویدانی

پدید آورندگان :

آیلین شاهرخ، مهشید زیوری

شماره دانشجویی :

40312358020، 40312358017

تابستان 1405

لینک گیت ہاب:

<https://github.com/aylilin/HoodQuest>

مقدمه :

پروژه HoodQuest یک بازی متنی مبتنی بر ساختمان داده ها و الگوریتم های گراف است که در محیط خط فرمان (CLI) اجرا میشود.

در این بازی، بازیکن نقش شنل قرمزی را برعهده دارد و باید در یک گراف از نقطه شروع خود حرکت کرده و به خانه مادر بزرگ برسد. در همین زمان، گرگ نیز در گراف حرکت میکند و تلاش میکند خود را به بازیکن برساند.

ساختار اصلی نقشه به صورت یک گراف وزن دار پیاده سازی شده است. رأس های گراف مکان های مختلف بازی و یال ها مسیر های قابل عبور بین این مکان ها را نشان میدهند.

هدف اصلی پروژه ، استفاده عملی از ساختمان داده ها و الگوریتم های مختلف در یک برنامه کاربردی است. در پروژه از ساختمان داده هایی مانند BST، Stack، Hash Table، Graph، Max/Min Heap، Heap و Queue استفاده شده و الگوریتم های Dijkstra، BFS و A* نیز پیاده سازی شده اند.

پروژه به چند بخش اصلی تقسیم شده است که به صورت پوشه های زیر میباشد :

- Algorithms
- DataStructures
- Game
- Models
- IO
- Maps

هدف از این تقسیم بندی، جدا کردن بخش مربوط به ساختمان داده ها، الگوریتم ها، منطق بازی، مدل های بازی و ورودی/خروجی است.

گراف بازی :

ساختار اصلی بازی یک گراف وزن دار است. در این گراف :

- هر رأس نشان دهنده یک موقعیت در نقشه است.
- هر یال نشان دهنده امکان حرکت بین دو رأس است.
- وزن هر یال هزینه یا فاصله آن مسیر را مشخص میکند.
- بازیکن و گرگ روی رأس های گراف قرار میگیرند.

ساختار نقشه در طول اجرای بازی ثابت باقی می ماند و بازیکن فقط میتواند به رأس هایی حرکت کند که با موقعیت فعلی او یال مستقیم دارند. این موضوع مطابق قوانین پروژه است که مسیر های قابل استفاده را بر اساس ساختار گراف تعیین میکند.

کلاس Graph وظیفه نگهداری رأس ها و ارتباط میان آنها را بر عهده دارد و متد هایی برای دریافت رأس ها و همسایه های هر رأس دارد.

ساختمان داده Hash Table :

برای مدیریت کاربران از یک Hash Table اختصاصی استفاده شده است. هر کاربر دارای اطلاعاتی مانند : Username، Password Hash و Score است. هنگام ثبت نام، نام کاربری در Hash Table بررسی میشود تا از تکراری نبودن آن اطمینان حاصل شود. هنگام ورود نیز ابتدا نام کاربری در Hash Table جستجو شده و سپس رمز عبور وارد شده با مقدار Hash شده ذخیره شده مقایسه میشود.

در پیاده سازی Hash Table از bucket و linked list برای مدیریت برخورد ها استفاده شده است.

عملیات هایی مانند : Insert – Contains-Get-Remove در این ساختمان داده پیاده سازی شده اند.

Undo و قابلیت Stack :

برای پیاده سازی قابلیت Undo از Stack استفاده شده است. قبل از انجام هر حرکت، وضعیت فعلی بازی در Stack ذخیره میشود. این وضعیت شامل اطلاعات مربوط به موقعیت بازیکن، موقعیت گرگ، امتیاز و وضعیت حرکت بازی است.

اگر بازیکن گزینه Undo را انتخاب کند، آخرین وضعیت از Stack خارج شده و وضعیت قبلی بازی بازیابی میشود.

پس از انجام Undo، دو امتیاز به عنوان جریمه از امتیاز بازیکن کم میشود.

این روش مطابق مشخصات پروژه است که ذخیره وضعیت قبل از هر نوبت و بازیابی آخرین وضعیت با Stack را مشخص کرده است.

u میتواند از قابلیت Undo استفاده کند.

ساختمان داده Queue :

ساختمان داده Queue در پیاده سازی الگوریتم BFS استفاده شده است.

این ساختمان داده باعث میشود رأس هایی که باید بررسی شوند به ترتیب ورود پردازش شوند.

رأس شروع وارد Queue می شود.

اولین رأس Queue خارج می شود.

همسایه های بازدید نشده آن بررسی میشوند.

همسایه ها وارد Queue میشوند.

این روند تا رسیدن به مقصد ادامه پیدا میکند.

الگوریتم BFS :

این الگوریتم در پروژه برای حرکت گرگ استفاده شده است. بعد از حرکت بازیکن، یک تاس شش وجهی انداخته می شود. اگر عدد تاس زوج باشد، گرگ مجاز به حرکت است و BFS کوتاه ترین مسیر بدون در نظر گرفتن وزن یال ها را از موقعیت گرگ تا موقعیت بازیکن پیدا میکند. سپس گرگ فقط یک یال در آن مسیر حرکت میکند. اگر تاس فرد باشد، گرگ در موقعیت خود باقی میماند. در کلاس BFS دو قابلیت اصلی وجود دارد :

`shortestPath()`

`nextMove()`

متد `shortestPath` مسیر کوتاه تر را پیدا کرده و `nextMove` اولین حرکت گرگ در آن مسیر را مشخص میکند.

الگوریتم Dijkstra :

این الگوریتم برای پیدا کردن کوتاه ترین مسیر وزن دار از موقعیت فعلی بازیکن تا خانه مادر بزرگ استفاده شده است.

در ابتدای هر نوبت، Dijkstra اجرا می شود و مسیر پیشنهادی به بازیکن نمایش داده میشود.

برای پیاده سازی Dijkstra از MinHeap استفاده شده است.

مرحل کلی الگوریتم :

1. فاصله رأس شروع برابر صفر قرار میگیرد.
2. رأس شروع در Min Heap قرار میگیرد.
3. رأس با کمترین فاصله از Heap خارج میشود.
4. یال های متصل به آن بررسی میشوند.
5. در صورت پیدا شدن مسیر کوتاه تر، فاصله رأس به روز رسانی میشود.
6. رأس بعدی در Heap قرار میگیرد.

این فرآیند تا رسیدن به مقصد ادامه پیدا میکند.

با استفاده از parent مسیر نهایی بازسازی میشود.

در بازی مسیر به صورت زیر نمایش داده میشود :

Suggested path (Dijkstra) : M -> S -> U -> V

Suggested next move : S

Min Heap :

برای اجرای الگوریتم های Dijkstra و A^* از Min Heap استفاده شده است.
در Min Heap، عنصری که کمترین Priority را دارد در بالای Heap قرار میگیرد.
در Dijkstra، Priority نشان دهنده فاصله فعلی از رأس شروع است.
در A^* ، Priority برابر مجموع هزینه طی شده و مقدار Heuristic است.
بنابراین Min Heap امکان انتخاب سریع رأس با کمترین مقدار Priority را فراهم میکند.

Max Heap و سیستم رتبه بندی :

برای نگهداری رتبه بندی بازیکنان از Max Heap استفاده شده است. در این ساختمان داده :
کلید، نام کاربر است و Priority امتیاز کاربر است.
بنابراین کاربری که بیشترین امتیاز را داشته باشد در بالای Max Heap قرار میگیرد.
در کلاس ScoreManager، یک Max Heap به نام ranking نگهداری میشود.
هنگام تغییر امتیاز کاربر، مقدار جدید هم در BST و هم در Max Heap ثبت میشود. در نتیجه
اطلاعات بازیکن برتر بدون نیاز به ساختن مجدد Heap قابل دسترسی است.
همچنین برای نمایش Top N بازیکنان، از یک کپی از Heap استفاده میشود تا Heap اصلی تغییر
نکند.

BST و مدیریت امتیاز ها :

برای نگهداری امتیاز کاربران از Binary Search Tree استفاده شده است. در BST:

Key -> Username

Value -> Score

قرار میگیرد. این ساختار امکان جستجوی امتیاز یک کاربر را فراهم میکند.

در برنامه، گزینه ای برای جستجوی امتیاز بازیکن وجود دارد و کاربر میتواند Username را وارد کرده و امتیاز خود را مشاهده کند.

همچنین از پیمایش Inorder برای دریافت اطلاعات مرتب شده امتیاز ها استفاده شده است.

A* و بخش امتیازی پروژه :

در پروژه الگوریتم A* نیز پیاده سازی شده است. برای پیدا کردن مسیر از موقعیت فعلی بازیکن تا خانه مادر بزرگ از این الگوریتم استفاده میشود.
ساختار کلی A* شامل دو مقدار است :

$g(n)$

هزینه مسیر طی شده از نقطه شروع تا رأس فعلی و :

$h(n)$

تخمین هزینه باقی مانده تا مقصد

در این پروژه برای محاسبه Heuristic ابتدا فاصله براساس تعداد یال ها با BFS محاسبه شده و سپس این مقدار در کمترین وزن یال موجود در گراف ضرب میشود. در نهایت :

$$f(n) = g(n) + h(n)$$

مبنای Priority در Min Heap قرار میگیرد.

کاربر میتواند در زمان اجرای بازی با وارد کردن : a مسیر پیشنهادی A* را مشاهده کند.

سیستم بازیکن :

کلاس Player اطلاعات مربوط به بازیکن را نگهداری میکند. مهم ترین اطلاعات بازیکن عبارتند از : شناسه بازیکن، موقعیت فعلی در گراف و امتیاز.

بازیکن در هر نوبت تنها میتواند به یکی از رأس های مجاور موقعیت فعلی خود حرکت کند.

اگر بازیکن رأس غیر مجاور را انتخاب کند، حرکت نامعتبر بوده و وضعیت بازی تغییر نمیکند.

سیستم گرگ:

کلاس Wolf موقعیت فعلی گرگ را نگهداری میکند. در هر نوبت پس از حرکت بازیکن، تاس انداخته میشود. اگر تاس زوج باشد:

ابتدا BFS، سپس پیدا کردن کوتاه ترین مسیر تا بازیکن و سپس حرکت یک یال.

اگر تاس فرد باشد:

گرگ حرکت نمیکند.